SISTEMA DE GESTIÓN

COLEGIO REINA DE LA ESPERANZA

Documentación técnica y manual de usuario

Dato	Información
Institución	Colegio Reina de la Esperanza
Nombre del sistema	Gestión Escolar Reina de la Esperanza
Tipo de solución	Aplicación web con frontend y API REST
Integrantes y roles	Tomas Agustin Lazarte - Desarrollador
Asignatura / docente	Proyecto Final de Programación
URL de demostración	https://colegio-demo-web.onrender.com

Índice

1. Introducción y descripción general

1. Introducción

2. Objetivos para cumplir

2. Diagramas

1. Infraestructura

2. Entidad-relación

3. Manual técnico

1. Preparación del proyecto

2. Front-end

3. Back-end

4. Despliegue y pruebas

4. Manual de usuario

1. Acceso al sistema

2. Vista del rol administrativo

3. Vista del rol profesor

4. Vista del rol alumno

5. Gestión de materias y horarios

6. Sistema de tareas y calificaciones

7. Control de asistencia

8. Sistema de comunicados

5. Conclusión

1. Introducción y descripción general

1.1 Introducción

Gestión Escolar Reina de la Esperanza es una aplicación web para apoyar la gestión académica y administrativa del Colegio Reina de la Esperanza. Reúne en una misma plataforma herramientas para el personal administrativo, los docentes y los alumnos, con acceso y pantallas diferenciadas por rol.

El sistema cuenta con una presentación institucional pública y módulos privados para tareas escolares como la gestión de usuarios, materias y clases, el seguimiento de tareas y entregas, la consulta de calificaciones, la asistencia y la publicación de anuncios.

La solución se compone de una interfaz web construida con React y una API REST construida con Express. Los datos se almacenan en una base relacional: MariaDB se utiliza como configuración predeterminada para el desarrollo local y PostgreSQL está habilitado para el despliegue de demostración.

Usuarios del sistema

Usuario	Uso principal
Administrativo	Administra usuarios, materias, clases y procesos académicos habilitados para su rol.
Profesor	Consulta clases y alumnos, publica tareas y anuncios, y trabaja con entregas, notas y asistencia.
Alumno	Consulta información académica, tareas, asistencia, boletín y anuncios; puede entregar trabajos.

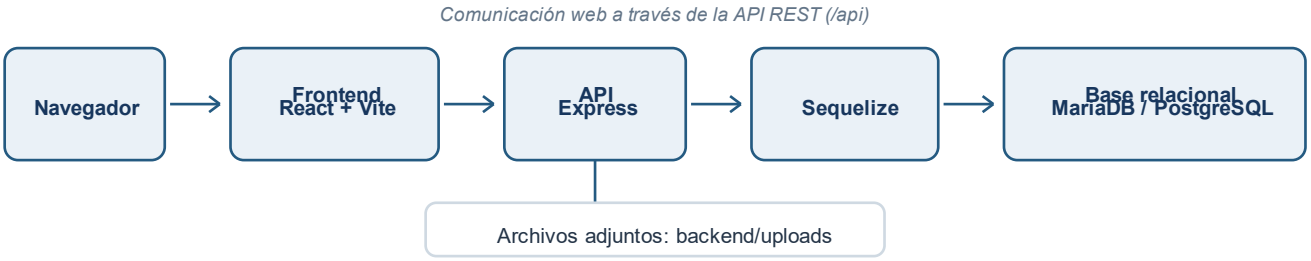
1.2 Objetivos para cumplir

- **Modernizar la gestión académica:** digitalizar procesos escolares y facilitar el acceso a la información.
- **Mejorar la interacción:** ofrecer herramientas diferenciadas para alumnos, docentes y personal administrativo.
- **Centralizar la administración:** gestionar cuentas, materias, clases y asignaciones desde la plataforma.
- **Acompañar el seguimiento académico:** facilitar el trabajo con tareas, entregas, calificaciones y asistencia.
- **Fortalecer la presencia institucional:** presentar al colegio y su propuesta educativa mediante una página pública.
- **Simplificar procesos escolares:** apoyar el registro de asistencia y la publicación de anuncios desde los módulos correspondientes.

2. Diagramas

2.1 Infraestructura

El navegador carga la aplicación frontend. Esta se comunica por HTTP/HTTPS con la API, que valida la sesión, ejecuta la lógica del servidor y accede a la base de datos. Los archivos adjuntos se sirven desde el backend.



Componentes

Componente	Responsabilidad
Frontend	Presenta las páginas institucionales y los paneles según el rol.

API REST	Expone operaciones bajo `/api`, valida solicitudes y aplica la lógica del servidor.
Sequelize	Capa de acceso relacional utilizada por el backend.
MariaDB / PostgreSQL	Persisten usuarios y registros escolares, según el entorno configurado.
Directorio `uploads`	Conserva los archivos adjuntos del backend; en el plan gratuito de Render el disco puede ser efímero.

2.2 Entidad-relación

El siguiente diagrama: resume entidades y asociaciones principales del esquema SQL

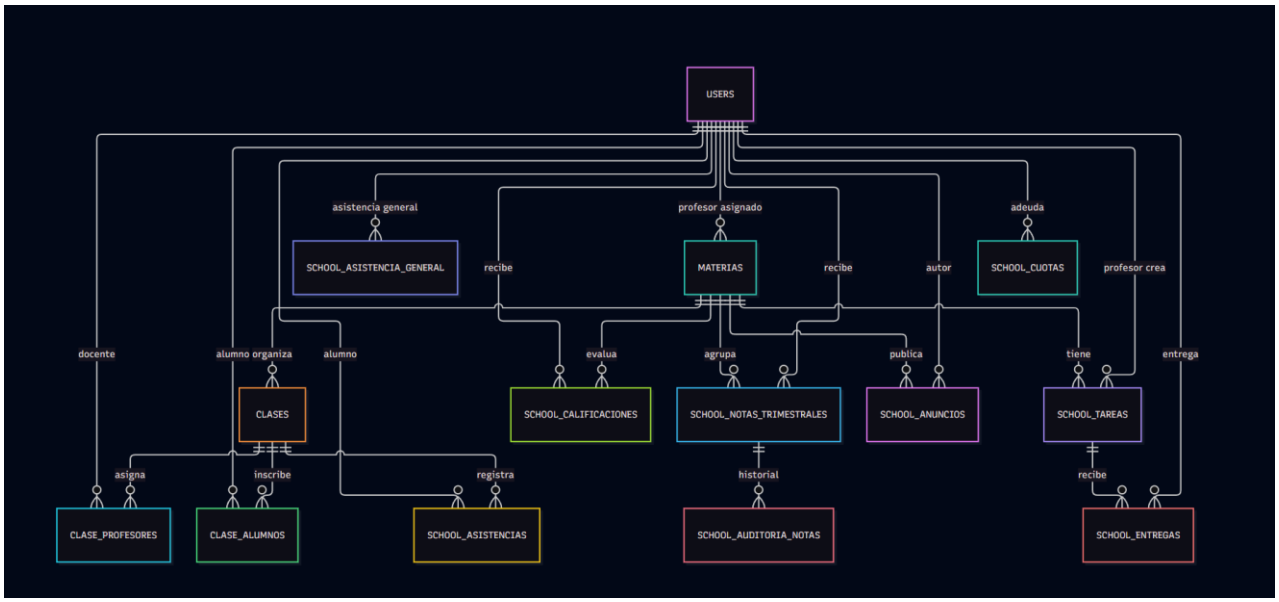


Tabla o grupo	Propósito
`users`	Cuentas, rol y datos escolares del usuario.
`materias`, `clases`	Materias, curso, división y horario.
`clase_profesores`, `clase_alumnos`	Asignaciones de docentes y alumnos a clases.
`school_tareas`, `school_entregas`	Tareas y entregas de los alumnos.
`school_asistencias`, `school_asistencia_general`	Asistencia asociada a clases y registro general.
`school_calificaciones`, `school_notas_trimestrales`	Calificaciones y notas por trimestre.
`school_auditoria_notas`	Historial de modificaciones de notas.
`school_anuncios`	Comunicados asociados a materias y docentes.
Tabla o grupo	**Propósito**
`school_cuotas`	Cuotas asociadas a usuarios.

3. Manual técnico

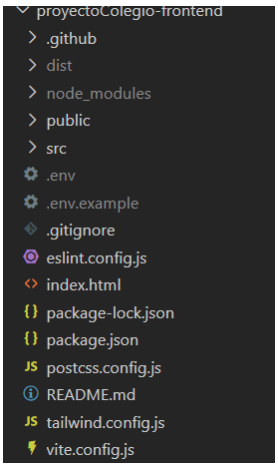
3.1 Preparación del proyecto

3.1.1 Cómo empezar

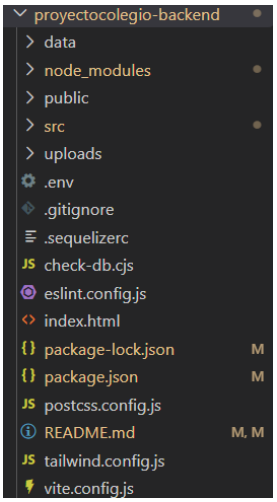
Requisitos: Node.js, npm, Git y una instancia MariaDB para el entorno local. PostgreSQL se usa en la configuración de despliegue de demostración.

El repositorio tiene dos aplicaciones independientes:

```
proyectoColegio-frontend/   Interfaz web
```




```
React/Vite proyectocolegio-backend/
```



3.1.2 Instalar las dependencias

Desde PowerShell, instalar las dependencias de cada aplicación:

```
cd proyectocolegio-
backend npm install
```

En otra terminal, desde la carpeta raíz:

```
cd proyectoColegio-
frontend npm install
```

Los comandos de desarrollo son `npm run dev` tanto en el backend como en el frontend. Ejecutarlos en terminales separadas.

3.1.3 Configurar la base de datos

Desarrollo local con MariaDB

1. Instalar/iniciar MariaDB y preparar credenciales locales.
2. En la terminal del backend, crear la base de datos y aplicar las migraciones académicas:

```
npm run db:create:local
npm run migrate:academics
```

3.1.4 Configurar las variables de entorno

Crear `proyectocolegio-backend/.env` localmente y completar valores propios. Ejemplo de desarrollo:

```
NODE_ENV=development
DB_DIALECT=mariadb
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=colegio
DB_USER=root
DB_PASSWORD=root
JWT_SECRET=colegio_reina_2025_ultrasecreto
FRONTEND_URL=http://localhost:5173
PORT=5000
```

3.2 Front-end

3.2.1 Tecnologías usadas

- **React:** construcción de la interfaz y componentes.
- **Vite:** servidor de desarrollo y empaquetado de producción.
- **React Router:** navegación entre páginas y paneles.
- **Axios:** solicitudes HTTP a la API REST.
- **Tailwind CSS:** utilidades de estilo.
- **Framer Motion:** animaciones de interfaz.
- **Context API:** estado compartido de autenticación.
- **JWT y almacenamiento del navegador:** el cliente adjunta el token a solicitudes autenticadas.

3.2.2 Implementación del código

El frontend está en `proyectoColegio-frontend/src/`. `App.jsx` configura las rutas; `AuthContext` mantiene el estado de autenticación; `PrivateRoute` restringe páginas a usuarios autenticados y, cuando se especifica, a roles permitidos.

Carpeta / archivo	Responsabilidad
`src/pages/`	Páginas y paneles de administración, docente y alumno.
`src/components/`	Componentes reutilizables, navegación, acceso y formularios.
`src/sections/`	Secciones de la página institucional pública.
`src/context/AuthContext.jsx`	Contexto de sesión y usuario.
`src/api/api.js`	Cliente Axios, URL del backend y funciones de acceso a endpoints.

La interfaz no sustituye la autorización del servidor: las operaciones protegidas deben comprobarse en la API y no solo ocultarse visualmente en el cliente.

Comandos habituales:

```
npm run
dev npm
run lint
npm run
build
```

3.3 Back-end

3.3.1 Tecnologías usadas

- **Node.js:** ejecución de JavaScript en el servidor.
- **Express:** API REST y montaje de rutas.
- **Sequelize:** acceso relacional y conexión MariaDB/PostgreSQL.
- **MariaDB / PostgreSQL:** almacenamiento de datos según el entorno.
- **JWT:** autenticación de solicitudes.
- **bcryptjs:** hash de contraseñas.
- **express-validator:** validación de campos en rutas que la implementan.
- **CORS, Helmet y Morgan:** configuración de solicitudes entre orígenes, cabeceras HTTP y registro de solicitudes.
- **Multer:** recepción de archivos.

3.3.2 Implementación del código

El backend está en `proyectocolegio-backend/src/`. Sigue una separación por rutas, controladores, middlewares, modelos/configuración y migraciones:

Carpeta / archivo	Responsabilidad
`src/index.js`	Configura Express, middleware, rutas `/api` y el inicio del servidor.
`src/routes/`	Declara endpoints y middleware de acceso/validación.
`src/controllers/`	Implementa operaciones solicitadas por las rutas.
`src/middlewares/`	Autenticación, permisos y validaciones compartidas.
`src/models/`	Modelos de datos y asociaciones usadas por el backend.
`src/config/mariadb.js`	Selección del dialecto y conexión Sequelize.
`src/database/migrations/`	Esquema relacional versionado.
`uploads/`	Archivos adjuntos publicados desde `/uploads`.

Los principales prefijos de API son `/api/auth`, `/api/usuarios`, `/api/materias`, `/api/clases`, `/api/tareas`,

`/api/entregas`, `/api/calificaciones`, `/api/notas`, `/api/asistencias`, `/api/anuncios`, `/api/cuotas` y `/api/admin`. La ruta `/` comprueba que la API inició y muestra el dialecto conectado.

Comandos habituales:

```
npm run dev
npm test
npm run test:postgres-compat
```

3.3.3 Configuración de correo y archivos

La interfaz incluye solicitud y restablecimiento de contraseña. El envío real de enlaces por correo depende de una configuración SMTP del entorno; verificarla antes de presentar la recuperación como operativa.

Los adjuntos se guardan en el directorio `uploads` del backend y se acceden mediante `/uploads`.

4. Manual de usuario

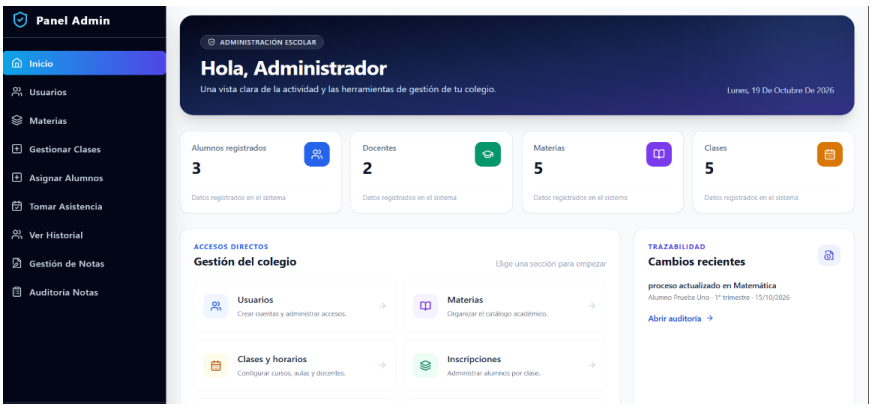
4.1 Acceso al sistema

1. Abrir la URL de la aplicación web proporcionada por la institución.
2. Seleccionar el acceso e ingresar el correo y la contraseña asignados.
3. Si las credenciales son válidas, el sistema dirige al usuario al panel de su rol: administración, profesor o alumno.
4. Si no puede iniciar sesión, comprobar los datos con el responsable del sistema. El flujo de recuperación permite solicitar un restablecimiento cuando el correo está configurado.

4.2 Vista del rol administrativo

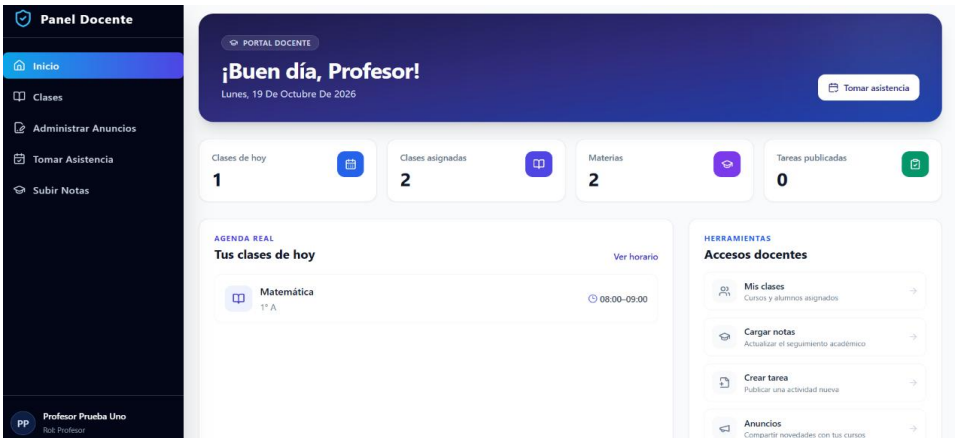
El panel administrativo incluye accesos a funciones como:

- Resumen del panel administrativo.
- Consulta y gestión de usuarios.
- Gestión de materias y clases.
- Asignación de alumnos a clases.
- Registro/consulta de asistencias.
- Gestión, revisión y auditoría de notas.



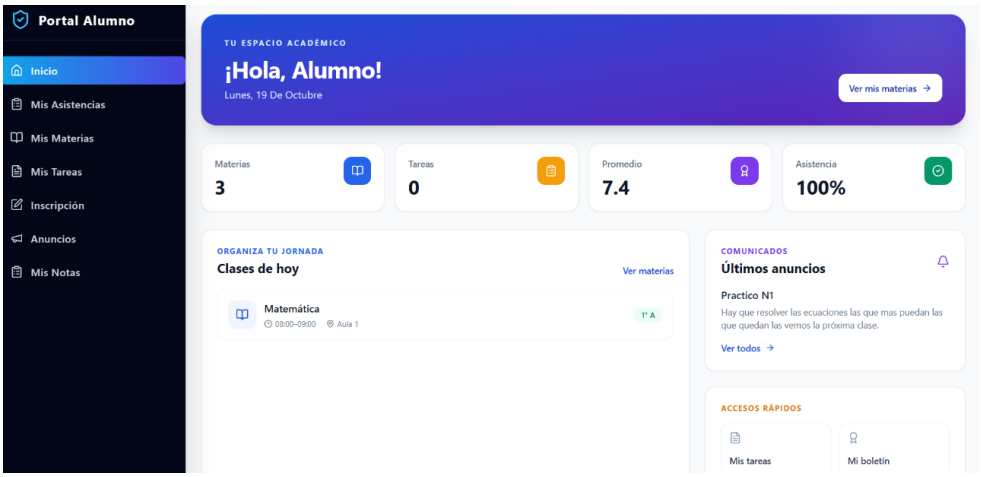
4.3 Vista del rol profesor

El panel docente dispone de páginas para consultar clases, alumnos y tareas, cargar tareas, revisar entregas, gestionar anuncios, tomar asistencia y trabajar con las notas de sus materias.



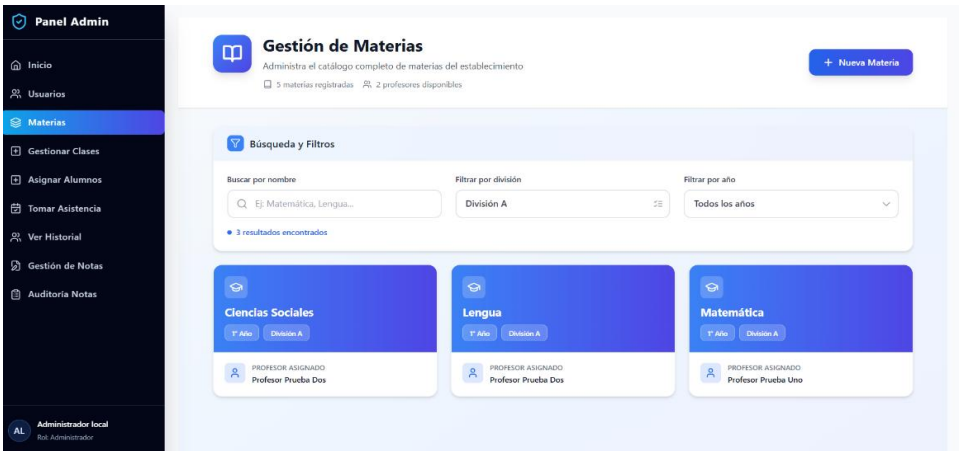
4.4 Vista del rol alumno

El panel del alumno ofrece accesos a inicio, materias, inscripción/consulta de clases, tareas, asistencia, boletín y anuncios. Desde los flujos correspondientes puede consultar actividades y enviar una entrega.



4.5 Gestión de materias y horarios

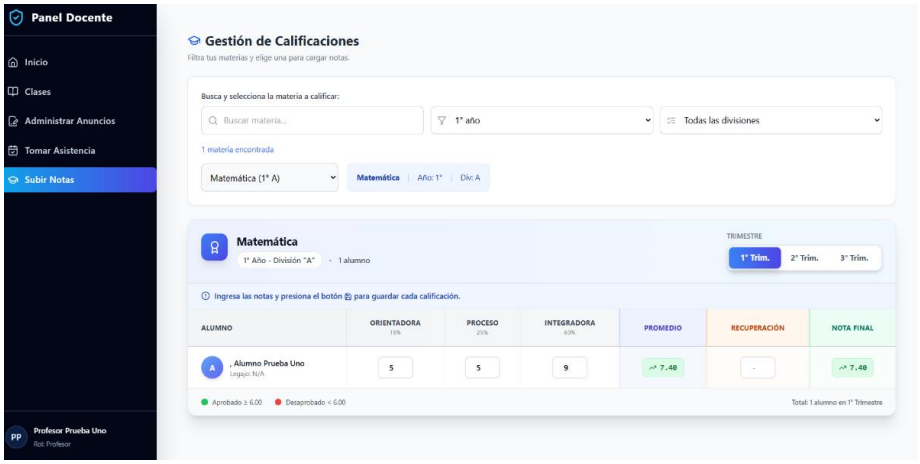
El sistema organiza materias y clases con datos como año, división, día, hora y aula. El personal autorizado administra las materias y clases, y gestiona las asignaciones de docentes y alumnos disponibles en el sistema.



4.6 Sistema calificaciones

El flujo general de tareas es:

1. El profesor sube sus notas por etapas y el sistema realiza un promedio general.
2. Las notas son controladas por el administrador.
3. Permite llevar un control con mucha transparencia.



4.7 Control de asistencia

Los usuarios autorizados pueden registrar asistencia de alumnos asociada a una clase y fecha. El sistema contempla estados como presente, ausente, tarde y justificado en las tablas de asistencia. El alumno puede consultar su asistencia desde su panel.

4.8 Sistema de comunicados

El módulo de anuncios permite al profesor publicar comunicados asociados a materias; los alumnos cuentan con una página para consultar anuncios. Revisar la audiencia que aplica al anuncio en la implementación desplegada antes de describirlo como institucional o personalizado.

5. Conclusión

El Sistema de Gestión Escolar Reina de la Esperanza representa una solución integral para la modernización de la administración académica. Este proyecto trasciende las limitaciones de los sistemas tradicionales de gestión educativa, proporcionando una plataforma digital que unifica todos los aspectos de la vida escolar.

La implementación de tecnologías modernas como React.js, Node.js y MongoDB garantiza un sistema robusto, escalable y de fácil mantenimiento. La arquitectura basada en roles asegura que cada usuario acceda únicamente a las funcionalidades relevantes para su función dentro de la institución.

El alcance del proyecto abarca desde la gestión básica de usuarios hasta funcionalidades avanzadas como el seguimiento académico personalizado y la comunicación institucional. Cada componente ha sido diseñado pensando en la experiencia del usuario final, priorizando la usabilidad y la eficiencia.

Este sistema no solo digitaliza los procesos existentes, sino que los optimiza, reduciendo la carga administrativa y permitiendo que el personal educativo se enfoque en lo que realmente importa: la educación de calidad. Además, la presencia web institucional fortalece la imagen del colegio y facilita la comunicación con las familias.